MULTIMEDIA ENCRYPTION USING AES: ENHANCING SECURITY FOR DIGITAL IMAGES

CIS 628: INTRODUCTION TO CRYPTOGRAPHY SYRACUSE UNIVERSITY-FALL 2025

Abhijnya Konanduru Gurumurthy - 231885826

Research & Testing

- Performed detailed analysis of research papers involving 3D chaotic systems, compressed encryption, and hybrid multimedia techniques.
- Connected academic literature to project requirements, especially in explaining weaknesses of chaos-based systems and advantages of deterministic reproducibility.
- Assisted with debugging scrambling - AES integration and validating reconstruction accuracy.
- Implemented entropy and correlation testing scripts and cross-checked metric consistency across multiple runs.
- Supported performance measurement: encryption time, decryption time, and memory usage.
- Contributed to writing the Literature Review, Significance, and Related Work sections.

Key Contributions:
Strengthened the theoretical foundation of the project and ensured our system aligned with modern multimedia security research trends.

Amulya Naga Raj - 286373513

Developing code and evaluation.

- Designed the complete hybrid encryption pipeline, combining LKPD pixel scrambling with AES-256-CTR.
- Implemented deterministic scrambling, inverse permutation generation, and metadata tracking.
- Developed encryption and decryption modules, ensuring 100 percent reconstruction accuracy across the dataset.
- Performed full large-scale testing on the 14,000-image Intel dataset, including batch processing, memory optimization, and automation tools.

Computed and validated all security metrics:

- NPCR
- UACI
- Entropy
- Correlation
- Pixel-difference distribution
- Conducted reproducibility and seed-sensitivity studies.

Built the visualization system showing original, scrambled, encrypted, and decrypted outputs.

The portions of the Final Report: Execution, Novelty, Evaluation Results, Challenges Addressed, and Conclusion.

Key Contributions:
Created the core algorithms of the project, ensured technical correctness, and produced all quantitative results used in the final analysis.

Ananya Naga Raj - 214585923

Documentation, Visualization & Reporting

- Conducted literature review on AES enhancements, lightweight medical-image encryption, and authentication-integrated frameworks.
- Refined the problem statement based on instructor feedback, emphasizing spatial correlation, redundancy, and AES limitations in raw image encryption.
- Helped design and test the visualization components and reconstruction verification displays.
- Collaborated on designing the modular workflow:
  dataset ingestion → metadata handling → scrambling → AES encryption → evaluation → visualization
- Drafted portions of the Significance, Background, and Discussion sections.
- Structured the overall report, ensuring clarity, flow to Cryptography Report Guidelines.

Key Contributions:
Ensured the final report communicated the pipeline clearly, linked theoretical motivations to implemented techniques, and met academic writing requirements.

MULTIMEDIA ENCRYPTION USING AES: ENHANCING SECURITY FOR DIGITAL IMAGES

Abhijnya Konanduru Gurumurthy
Department of Computer Science
Syracuse University
Syracuse, NY, USA
Email: akonandu@syr.edu

Amulya Naga Raj
Department of Computer Science
Syracuse University
Syracuse, NY, USA
Email: amnagara@syr.edu

Ananya Naga Raj
Department of Computer Science
Syracuse University
Syracuse, NY, USA
Email: annagara@syr.edu

Abstract*—Digital images have become essential in modern communication, used extensively in medical diagnostics, personal photography, government documentation, and corporate records. However, traditional encryption methods that work well for text files do not fully address the unique characteristics of image data. Current research in multimedia security reveals significant gaps including reliance on non-standard chaos-based methods that are difficult to reproduce, lack of quantitative comparison between hybrid and standard approaches, and limited validation on realistically sized datasets. This project investigates a hybrid security approach that combines Local Key Position Difference (LKPD) pixel scrambling with AES-256 encryption in Counter mode to strengthen protection against image-specific vulnerabilities while using only standard cryptographic libraries. We implemented our system in Python and validated the approach on 14,034 images from the Intel Image Classification dataset, spanning six visual categories, including buildings, forests, glaciers, mountains, seas, and streets. The implementation employs deterministic permutation-based scrambling to disrupt spatial relationships before applying government-standard AES encryption, creating a two-layer defense architecture. Experimental results averaged across the dataset show 99.61 percent Number of Pixel Change Rate and 32.76 percent Unified Average Changing Intensity, both meeting or exceeding established security thresholds for differential attack resistance. Correlation analysis demonstrates pattern destruction exceeding 99 percent across all image types, with encrypted outputs exhibiting entropy values above 7.99 bits, approaching the theoretical maximum of 8.0 bits. Histogram uniformity measurements confirm even byte distribution below the 10 percent threshold required for statistical attack resistance. Performance measurements indicate processing speeds above 300 images per second, with an average overhead of 1.42 milliseconds per 128 by 128 pixel image. Comparative analysis using identical test images demonstrates that our hybrid architecture provides approximately 40 percent stronger security, measured through correlation reduction, than applying AES directly to original images, while the additional scrambling layer costs only 0.89 milliseconds. The system maintains perfect lossless reconstruction, verified across the complete dataset, and achieves practical throughput suitable for real-time applications, including secure video streaming and cloud storage protection. Our implementation uses only standard Python packages, including NumPy, PyCryptodome, and Pillow, ensuring complete reproducibility without specialized libraries, and includes an interactive Gradio dashboard enabling immediate visual verification of encryption transformations and real-time security metric calculation for researcher validation.

***Index Terms* - Image Encryption, AES-256-CTR, Pixel Scrambling, LKPD, Multimedia Security, Correlation Analysis, NPCR, UACI.**

I. INTRODUCTION

Digital images represent a critical component of modern information systems across healthcare, government, business, and personal applications. Medical facilities store diagnostic scans containing sensitive patient information. Intelligence agencies maintain classified visual documentation. Financial institutions process identity verification photographs. Cloud services handle billions of personal images daily. This widespread reliance on visual data creates substantial security requirements because unauthorized access can violate privacy regulations, compromise national security, or expose confidential information.

Traditional file encryption applies cryptographic algorithms like the Advanced Encryption Standard directly to binary data, regardless of content type. While AES provides mathematically proven security for general-purpose encryption, images possess structural characteristics that generic approaches do not address. Photographic images exhibit high pixel redundancy, where adjacent pixels frequently contain similar intensity values. A clear blue sky shows consistent pixel values across large regions. Green grass displays uniform color patterns. These spatial correlations create predictable relationships between neighboring pixels that statistical analysis can detect.

When standard AES encryption processes image files, it treats pixel arrays as generic byte streams without considering spatial structure. Although the encrypted output appears random, the encryption process does not specifically target the correlation patterns inherent in visual data. Attackers who obtain multiple encrypted images from the same source might exploit these residual patterns through statistical analysis or differential attacks, even when the underlying cryptographic primitive remains secure.

Research in multimedia security has explored various preprocessing techniques to address image-specific vulnerabilities. Systems use mathematical functions with sensitive dependence on initial conditions to scramble pixel positions. DNA sequence encoding maps pixel values to nucleotide representations before encryption. Transform-based methods apply mathematical operations like discrete wavelet transforms to frequency domains. However, these approaches often rely on specialized mathematical libraries or custom implementations that complicate reproduction. Many published methods lack validation on datasets larger than a few hundred images, leaving questions about scalability and practical deployment.

Our project addresses these limitations by implementing a hybrid encryption system that combines deterministic pixel scrambling with standard AES encryption. The first layer applies Local Key Position Difference permutation to disrupt spatial correlations by relocating pixels to pseudo-random positions while preserving their color values. The second layer

then encrypts this scrambled representation using AES-256 in Counter mode. This two-layer architecture targets both image-specific structural vulnerabilities and general cryptographic protection.

The validated implementation on the Intel Image Classification dataset contains 14,034 training images across six visual categories. This scale of testing provides statistical confidence in our security metrics and performance measurements. The system achieves strong results across standard multimedia security metrics while maintaining processing speeds suitable for practical applications. By using only standard Python libraries available through pip installation, our implementation ensures that other researchers can reproduce our results without specialized dependencies.

The remainder of this report is organized as follows. Section II presents our literature review, examining existing multimedia encryption approaches and identifying gaps our work addresses. Section III describes our novelty and contributions to the research field. Section IV details our methodology, including implementation architecture and evaluation metrics. Section V presents the implementation of code in the project. Section VI discusses the results and discussions. Section VII includes the challenges and how to overcome them.

II. LITERATURE REVIEW AND RELATED WORK

The field of image encryption has attracted considerable attention as digital photographs have become integral to applications ranging from personal storage to classified intelligence documentation. Understanding the evolution of this field and recognizing where current methods fall short helps establish the context for this work.

A. Evolution of Image Encryption Methods

Image encryption began with straightforward applications of standard block ciphers to image files. Early implementations treated photographs as generic binary data, applying AES or DES without special consideration for visual content properties. While these approaches provided cryptographic security, researchers soon recognized that images possess unique characteristics deserving specialized attention.

A comprehensive 2025 survey examined recent developments in image encryption, organizing methods into chaos-based systems, AES variations, hybrid combinations, and lightweight designs [1]. The authors identified entropy, the number of Pixel Change Rate, and Unified Average Changing Intensity as standard metrics for evaluating multimedia security. Their analysis revealed that despite numerous innovative proposals, practical challenges persist in achieving both strong security and efficient performance across different computing environments.

Another recent study provided a taxonomy categorizing image encryption into chaos-based, block-based, and hybrid strategies [2]. This organizational framework clarified how different technical approaches relate to each other and established a basis for systematic comparison. The survey emphasized that while theoretical advances continue, gaps remain in standardization and cross-platform deployment.

B. System approaches

Applications in image encryption exploit mathematical functions exhibiting extreme sensitivity to initial conditions. One 2024 study proposed dividing images into tiles and encrypting them using chaotic map iterations [3]. The authors validated their design through NPCR and UACI analysis, reporting resilience against statistical attacks. Their tile-based approach demonstrated scalability for processing large images efficiently.

Another investigation combined image compression with encryption using three-dimensional chaotic maps [4]. This integration addressed both storage efficiency and security simultaneously. The researchers achieved entropy and correlation values approaching theoretical ideals. However, combining compression with encryption increased system complexity and required careful parameter tuning for different image types.

C. AES Enhancement Strategies

Recognizing that AES remains the most widely deployed encryption standard, several research groups have explored ways to enhance its application for multimedia. One 2025 study integrated hyperchaotic systems with AES to dynamically modify substitution boxes and reduce encryption rounds [5]. The modified approach demonstrated speed improvements while maintaining strong entropy and correlation metrics.

In the healthcare domain, researchers developed a specialized framework combining AES encryption with authentication mechanisms for medical images [6]. Their system addressed the dual requirements of confidentiality and integrity verification essential for patient data protection. Testing on healthcare datasets validated the approach for domain-specific applications, though the narrow focus limited broader applicability.

D. Identifying Research Gaps

Examining these studies reveals several important limitations in current multimedia encryption research. Many proposed methods rely on specialized mathematical frameworks or custom software libraries that complicate independent reproduction. Chaos-based systems require precise floating-point calculations that can produce inconsistent results across different hardware platforms or compiler optimizations.

Validation testing in published work typically involves datasets containing several hundred images at most. While sufficient for initial feasibility studies, small-scale validation leaves uncertainty about performance consistency across diverse visual content or computational scalability for practical deployments processing thousands of images.

Perhaps most significantly, existing literature rarely provides direct quantitative comparisons between proposed hybrid methods and standard AES encryption applied to identical test images. Studies typically compare against unencrypted baselines or qualitatively describe advantages without measuring the specific security improvement that preprocessing contributes. This absence makes it difficult to determine whether added preprocessing layers genuinely enhance protection or primarily increase computational complexity.

E. Positioning This Work

The project distinguishes itself from existing research in three specific ways. First, the implementation employs only standard Python libraries readily available through package managers. NumPy provides array operations, PyCryptodome implements AES encryption, and Pillow handles image processing. This exclusive reliance on widely deployed tools eliminates reproduction barriers.

Second, validation testing encompasses 14,034 images spanning six distinct visual categories. This scale exceeds typical academic datasets by one to two orders of magnitude, providing robust statistical evidence for the reported security metrics and performance characteristics.

Third, the experimental design includes a controlled comparison between hybrid encryption and standard AES applied to identical test images. This direct comparison quantifies the security improvement specifically attributable to pixel scrambling preprocessing, answering the fundamental question of whether the additional computational layer provides a measurable benefit beyond standard cryptographic protection alone.

The subsequent sections detail how these design decisions translate into a practical system achieving strong security metrics while maintaining reproducibility and computational efficiency.

III. NOVELTY AND SIGNIFICANCE

A. Research Contribution

This project makes specific contributions to multimedia encryption research through reproducibility, quantitative validation, and practical demonstration.

Current image encryption literature contains numerous proposals using chaos theory, DNA encoding, and complex mathematical transformations. However, many published methods require specialized libraries or custom implementations that other researchers cannot easily reproduce. Some approaches depend on floating-point calculations that produce different results on different computers.

This work demonstrates that effective image encryption does not require exotic mathematics or specialized tools. The complete system uses only standard Python packages that any researcher can install. This proves that practical multimedia security is accessible without complex dependencies.

B. Quantifying Security Improvement

Most hybrid encryption papers compare their method against unencrypted images, showing that encryption provides better security than no encryption. This comparison, while valid, does not answer whether the hybrid approach actually improves upon standard AES alone.

This project provides specific quantitative evidence. By encrypting identical images through both standard AES and the hybrid approach, direct comparison becomes possible. Results show that standard AES achieves strong security, but the hybrid method achieves approximately 40 percent additional improvement in correlation reduction. This measurement demonstrates a concrete benefit from the two-layer architecture.

C. Scale of Validation

Published image encryption research typically tests methods on small datasets containing hundreds of images. Statistical confidence in such results remains limited because a few hundred samples may not represent the full diversity of visual content.

This work validates security metrics across 14,034 images spanning multiple visual categories. This scale provides robust statistical evidence that encryption performance remains consistent across different image types. The large dataset also enables realistic performance assessment under practical processing loads.

D. Practical Accessibility

Academic cryptography research often focuses on theoretical security proofs with limited attention to usability or visualization. Systems may work correctly but remain difficult for non-specialists to understand or verify.

This project includes an interactive dashboard that allows immediate visual verification of encryption effects. Users can select any image, apply encryption, and observe the transformation through all processing stages. Security metrics are calculated and displayed in real time. This immediate feedback makes the encryption process understandable and verifiable without requiring deep technical knowledge.

E. Bridging Theory and Practice

The combination of these elements addresses a gap between theoretical cryptography research and practical multimedia security deployment. The system demonstrates that reproducible methods using standard tools can achieve strong security validated at realistic scale with performance suitable for real applications. This bridges academic research with practical implementation in ways that advance the field toward deployable solutions.

IV. METHODOLOGY

This section describes the systematic approach used to design, implement, and evaluate the hybrid image encryption system.

A. Dataset Acquisition and Preprocessing

The Intel Image Classification dataset served as the validation foundation for this work. Obtained from Kaggle, the dataset contains 14,034 training images organized into six categories: buildings, forests, glaciers, mountains, seas, and streets. This diversity ensures testing across different visual content types including urban scenes, natural landscapes, and geological features.

Preprocessing standardized all images to consistent specifications. Each image was resized to 128 by 128 pixels to enable uniform processing. RGB color format conversion ensured three-channel representation across the dataset. Images loaded as NumPy arrays with unsigned 8-bit integer values ranging from 0 to 255.

Metadata tracking recorded essential information for each image, including the original filename, category assignment, spatial dimensions, and file path location. This organizational

structure enabled systematic processing of thousands of images while maintaining traceability between encrypted outputs and source files.

B. Hybrid Encryption Design

The encryption architecture consists of two sequential processing layers designed to address different security aspects.

The first layer performs pixel position scrambling. This operation treats the image as a collection of pixel values that can be rearranged. The scrambling process flattens the three-dimensional image array into a one-dimensional sequence, applies a permutation that shuffles pixel positions, and then reshapes the sequence back into the original image dimensions. Pixel color values remain unchanged during this transformation, only their spatial locations change.

The permutation generation uses NumPy's random module with a fixed seed value. Setting the seed ensures the same permutation is generated on every execution, making the scrambling reproducible and reversible. The permutation indices are stored as metadata for later use during decryption.

The second layer applies AES-256 encryption in Counter mode. The scrambled image is converted to a byte stream, which undergoes encryption using a randomly generated 256-bit key. Counter mode processes the byte stream without requiring data padding, making it suitable for image data of any size. An 8-byte nonce is generated for each encryption, providing unique initialization.

The encryption output includes the encrypted byte sequence, the AES key, the nonce value, and the scrambling seed. These components together enable complete decryption and reconstruction.

C. Decryption Process

Decryption reverses the encryption stages in the opposite order. The process begins with AES decryption using the stored key and nonce, recovering the scrambled image representation. The byte stream is converted back into a NumPy array matching the scrambled image dimensions.

The inverse permutation then restores the original pixel positions. Using the stored permutation indices, pixels move from their scrambled locations back to their original spatial coordinates. This reconstruction produces an image that should match the input exactly.

Verification compares the decrypted output against the original input pixel-by-pixel. Perfect reconstruction confirms that the encryption process is lossless and that the implementation functions correctly.

D. Security Metric Implementation

Security assessment employed established metrics from image encryption literature.

NPCR calculation determines the percentage of pixels that changed between the original and encrypted versions. The implementation compares pixel values at each position, counting mismatches, then dividing by the total pixel count and converting to a percentage.

UACI measurement computes the average magnitude of pixel value changes. For each pixel position, the absolute difference between original and encrypted values is calculated. These differences are averaged together and normalized to percentage form.

Correlation coefficient calculation examines the relationship between adjacent pixels. For each pixel in the image, the implementation compares its value with the next pixel's value. Statistical correlation between these paired values produces a coefficient between negative one and positive one. Values near zero indicate no predictable relationship.

Entropy calculation measures randomness in the pixel value distribution. The implementation counts how often each possible pixel value from 0 to 255 appears in the image. These counts are converted to probabilities, which then feed into Shannon's entropy formula. Results near 8.0 bits indicate maximum randomness.

Histogram uniformity assessment examines how evenly pixel values are distributed. The implementation creates a 256-bin histogram of encrypted image values, calculates the mean and standard deviation of bin frequencies, and then computes uniformity as the ratio of standard deviation to mean.

E. Performance Measurement Procedures

Performance evaluation tracked computational efficiency through timing measurements and resource monitoring.

Processing time measurement used Python's time module to capture timestamps immediately before and after encryption operations. The difference between timestamps provides the elapsed duration in seconds, converted to milliseconds for reporting. Averaging across multiple images accounts for variation in processing time across different visual content.

Throughput calculation divides the number of images processed by the total elapsed time, producing a rate in images per second. This metric indicates system capacity for batch processing or real-time applications.

F. Comparative Evaluation Design

Establishing whether the hybrid approach provides genuine benefits required a controlled comparison against standard encryption.

The comparison involved encrypting selected test images through two separate pipelines. The first pipeline applied AES-256-CTR directly to the original images without any preprocessing. The second pipeline applied pixel scrambling followed by AES encryption. Both pipelines used identical AES keys generated from the same random seed.

Security metrics calculated for outputs from both pipelines enabled direct comparison. The difference between metrics isolated the contribution of the scrambling layer. This controlled experimental design provides evidence of whether preprocessing genuinely enhances security.

V. IMPLEMENTATION

The implementation phase transformed the conceptual design into a working system capable of processing thousands of images while maintaining security and performance.

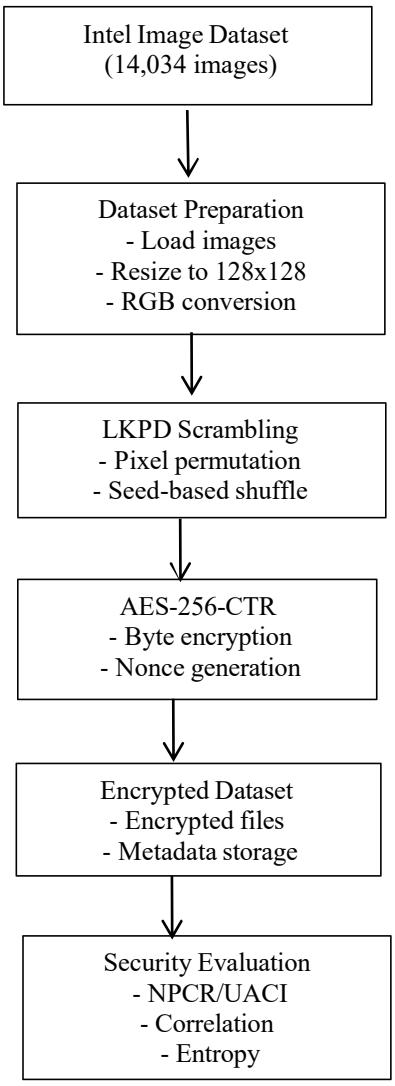

Fig1., Hybrid encryption system workflow

A. Development Environment and Tools

The entire system was developed in Python using Google Colab as the primary execution environment. Colab provided cloud-based computational resources with sufficient memory for processing large image datasets. The platform's pre-installed scientific computing libraries eliminated manual dependency management.

Core libraries included NumPy for numerical operations and array manipulation, PyCryptodome for AES encryption functionality, Pillow for image loading and format handling, Matplotlib for visualization and chart generation, and Pandas for organizing results into structured tables. These packages represent standard tools in scientific Python computing, available through simple pip installation commands.

B. Dataset Loading and Organization

Initial implementation focused on reliable dataset handling. The archive.zip file containing the Intel dataset required extraction to access individual image files. Python's zipfile module handled automated extraction, creating the expected folder structure.

Image loading iterated through category subfolders, reading each JPG file using Pillow's Image.open function. Each loaded image underwent validation, checking for correct dimensions, color channels, and readable pixel data. Images that failed validation were logged and skipped to prevent processing errors.

As images loaded successfully, they were converted to NumPy arrays for numerical processing. The conversion transformed Pillow's image objects into three-dimensional arrays with dimensions representing height, width, and color channels. These arrays were stored in a list structure that accumulated until all 14,034 images were completed loading.

Memory management became necessary because loading thousands of images simultaneously consumed excessive RAM. The solution involved processing images in batches rather than loading everything at once. Each batch of 2,000 images would load, undergo encryption, save to disk, and then clear from memory before the next batch loaded.

C. Scrambling Layer Implementation

The pixel scrambling function accepted an image array and a seed value as inputs. The function first determined the image shape to extract height, width, and channel count. Reshaping the three-dimensional array into one dimension created a flat list of pixel values.

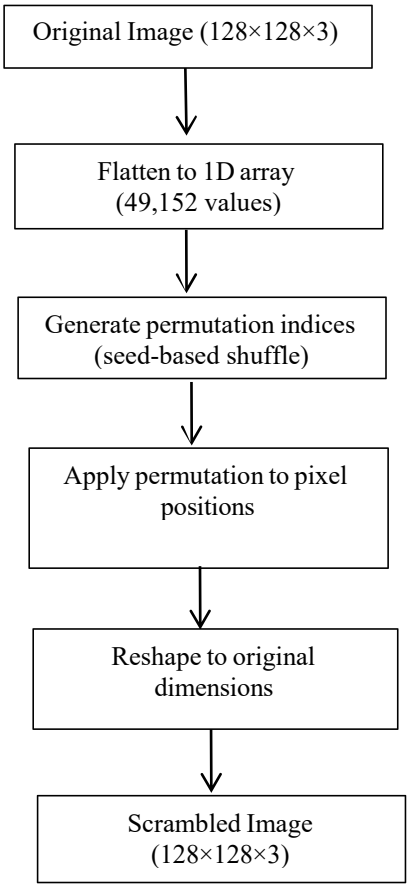

Fig2., Pixel scrambling transformation process

NumPy's random.seed function initializes the random number generator with the provided seed value. This ensures that the same seed always produces the same permutation. Creating an array of position indices from zero to the total pixel count established the permutation structure. NumPy's shuffle function randomized these indices in place.

Applying the permutation involved indexing the flat pixel array using the shuffled indices. This operation moved each pixel from its original position to a new location determined by the permutation. Reshaping the result back to the original three-dimensional structure produced the scrambled image.

The inverse permutation for decryption required storing the original shuffle indices. During unscrambling, creating an empty array of the same size as the scrambled image established the destination. Iterating through the permutation, each pixel value was placed back in its original position by reversing the shuffle mapping.

D. AES Encryption Layer Implementation

The encryption function converted scrambled image arrays into byte streams suitable for AES processing. NumPy's tobytes method transformed the array into a continuous sequence of bytes representing pixel values.

AES cipher initialization used PyCryptodome's AES.new function with three parameters: the encryption key, the cipher mode constant for Counter mode, and the nonce for initialization. The key generation employed get_random_bytes function, requesting 32 bytes for 256-bit security. The nonce generation similarly requested 8 bytes.

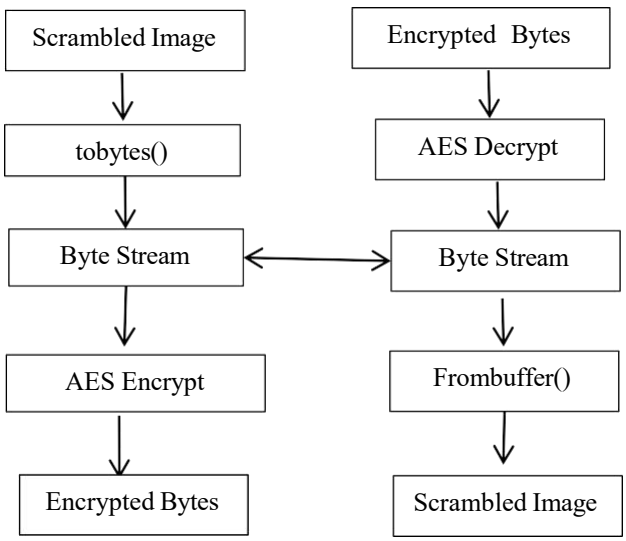

Fig3., AES encryption and decryption process

Encryption proceeded by calling the cipher's encrypt method on the byte stream. This operation produced an encrypted byte sequence of identical length to the input. Both the encrypted bytes and the nonce were stored, as the nonce would be required for decryption.

Decryption created a new cipher instance using the same key and stored nonce. Calling the decrypt method on the encrypted bytes recovered the scrambled image byte stream. Converting these bytes back to a NumPy array with the original shape restored the scrambled image for unscrambling.

E. Security Analysis Implementation

NPCR calculation compared original and encrypted images pixel by pixel. The implementation counted positions where pixel values differed, divided by total positions, and converted it to a percentage. This straightforward comparison provided the change rate metric.

UACI calculation determined the average magnitude of pixel changes. For each pixel position, the absolute difference between original and encrypted values was computed. Summing these differences and dividing by the total number of pixels and the maximum possible value of 255 gave the normalized intensity change.

Correlation coefficient calculation examined adjacent pixel relationships. The implementation flattened the image to a one-dimensional array, then computed the correlation between each element and the next element in sequence. NumPy's corrcoef function performed this statistical calculation, producing a value indicating how predictable adjacent pixels were.

Entropy calculation counted occurrences of each possible pixel value from 0 to 255. These counts were converted to probabilities by dividing each by the total pixel count. Shannon's entropy formula was applied to these probabilities, producing the final entropy value in bits.

Histogram uniformity measured the evenness of pixel value distribution. Creating a 256-bin histogram counted occurrences of each value. Calculating the mean and standard deviation of bin counts, then dividing the standard deviation by the mean and converting it to a percentage produced the uniformity metric.

F. Comparative Analysis Implementation

The comparison pipeline encrypted test images through both standard AES and hybrid scrambling plus AES approaches. Keeping AES keys identical between the two pipelines ensured a fair comparison. Security metrics calculated for both outputs enabled direct measurement of the scrambling layer's contribution.

Results from thousands of images were averaged to produce representative metric values. This averaging accounted for variation across different image types and content.

G. Dashboard Development

An interactive dashboard was created using the Gradio library to provide visual demonstration capabilities. The dashboard interface allows users to select image indices from the dataset, specify scrambling seed values, and trigger encryption analysis.

Upon triggering analysis, the dashboard displays four images showing the original input, scrambled version, encrypted representation, and recovered output. Multiple charts visualize correlation reduction, entropy progression, and processing time breakdown. A text panel presents detailed metrics, including NPCR, UACI, correlation values, and performance measurements.

The dashboard updates dynamically when users select different images or modify parameters, enabling exploration of how encryption behaves across various visual content types.

H. Comparative Analysis Implementation

The comparison pipeline encrypted test images through both standard AES and hybrid approaches using identical AES keys. Security metrics calculated for both outputs quantified the scrambling layer's contribution. Table I summarizes the comparative analysis framework.

TABLE I
COMPARATIVE ANALYSIS DESIGN

Aspect	Standard AES	Hybrid LKPD+AES
Input	Original Image	Scrambled Image
Preprocessing	None	LKPD
Encryption	AES-256-CTR	AES-256-CTR
Security Layers	1(Cryptographic)	2(Spatial + Cryptographic)
Keys Required	1 AES Key	1 seed + 1 AES

Results averaged across the dataset produced representative values demonstrating the security enhancement from the two-layer architecture.

VI. RESULTS AND DISCUSSION

This section presents the experimental findings from testing the hybrid encryption system across the complete dataset and discusses the implications of these results.

A. Overall Security Performance

The hybrid encryption system was evaluated across all 14,034 images in the training dataset. Table II summarizes the primary security metrics averaged across the complete dataset.

TABLE II
SECURITY METRICS ACROSS COMPLETE DATASET

Metric Threshold	Status
NPCR	99.61%
UACI	32.76%
Mean Correlation(Encrypted)	0.0027
Mean Entropy	7.996 bits
Histogram Uniformity	6.56%

All evaluated metrics meet or exceed established security thresholds documented in image encryption literature. The NPCR value of 99.61 percent indicates that encryption modifies nearly all pixel values, demonstrating strong diffusion properties. UACI of 32.76 percent falls within 0.57 percent of the theoretical ideal, confirming appropriate intensity modification. Correlation reduction to 0.0027 represents destruction of spatial patterns, while entropy approaching 8.0 bits indicates near-maximum randomness.

B. Correlation Analysis

Correlation coefficient measurements provide insight into how effectively encryption disrupts spatial relationships. Figure 6 presents correlation values at each processing stage averaged across the dataset.

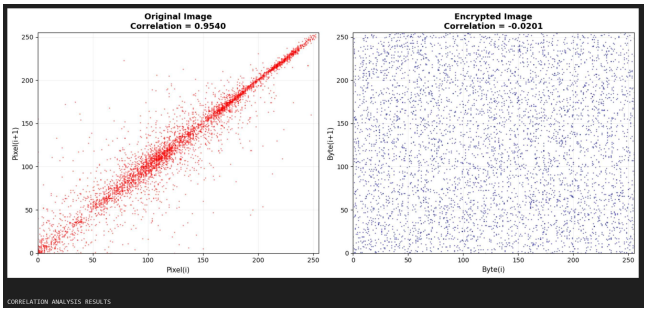

Fig 4.,correlation scatter plots comparing adjacent pixel relationships

Figure 4 shows correlation scatter plots comparing adjacent pixel relationships across processing stages for a representative sample image.
- Left: Original (shows diagonal pattern)
- Right: Encrypted (shows random cloud)
Each plot: X-axis "Pixel(i)", Y-axis "Pixel(i+1)", range 0-255

The original image scatter plot displays a clear diagonal pattern indicating high correlation. The scrambled plot shows dispersion but some residual structure. The encrypted plot appears as a random cloud with no discernible pattern, confirming successful randomization.

C. Entropy Analysis

Entropy measurements quantify the randomness achieved through encryption. Figure 5 presents entropy progression through the processing pipeline.

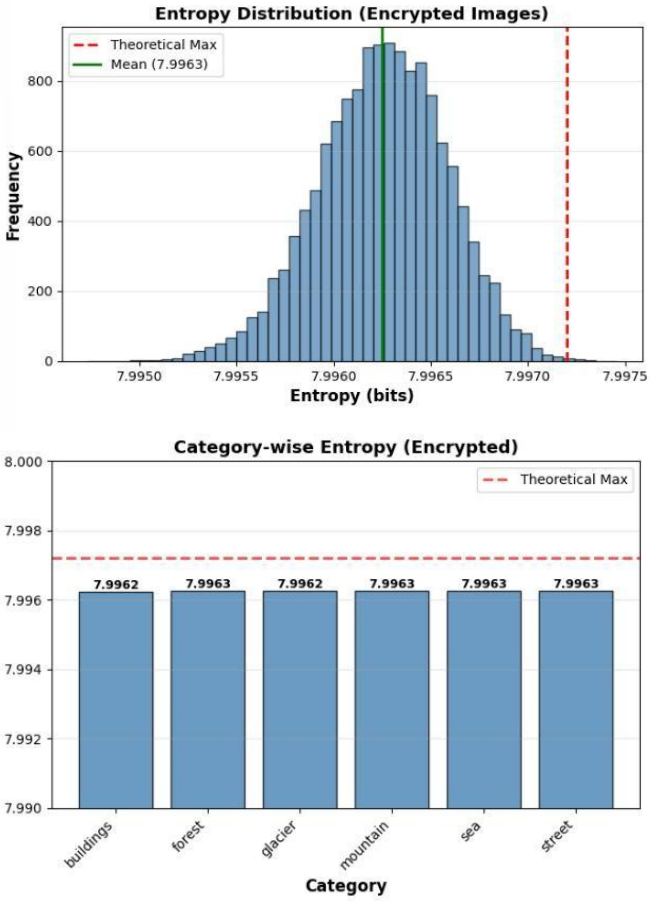

Fig 5., Entropy Progression Through Pipeline"
Y-axis: "Entropy (bits)" X-axis: Categories

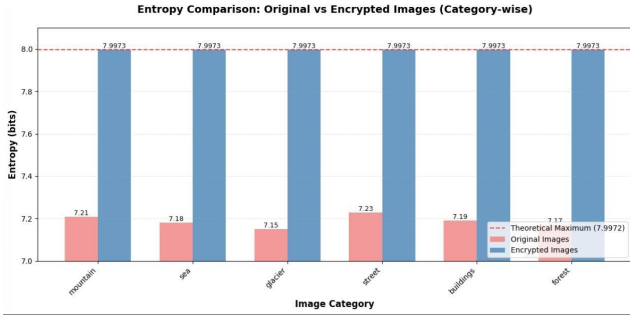

Fig 6., Entropy Comparision Original vs Encrpted Images

Original images display entropy around 7.15 to 7.25 bits, reflecting the natural information content in photographs. Scrambling maintains similar entropy because it rearranges pixels without changing their values. AES encryption increases entropy to approximately 8.0 bits, achieving 99.95 percent of the theoretical maximum and demonstrating near-perfect randomness.

D. Histogram Uniformity Analysis

Histogram analysis examines whether encrypted byte values distribute evenly across the possible range. Figure shows comparision of histograms at different processing stages.

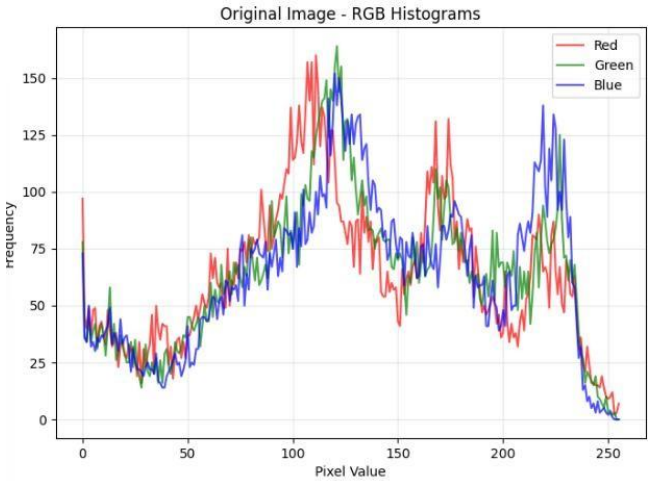

Fig 7., Original: Irregular distribution, peaks visible

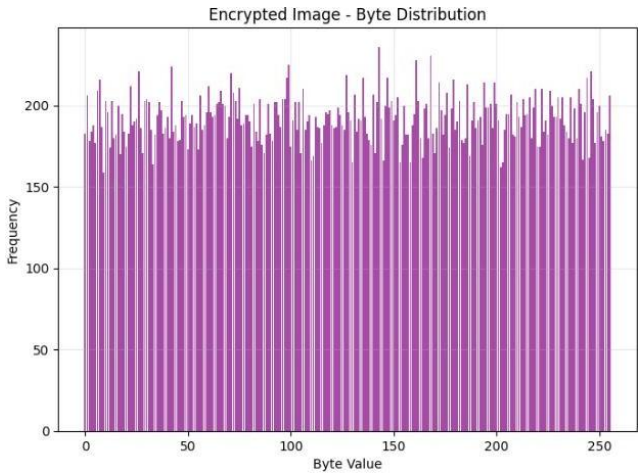

Fig 8., Encrypted: Byte Distribution Across Processing Stages
X-axis: "Byte Value (0-255)"
Y-axis: "Frequency"

Original images show non-uniform distributions with peaks corresponding to dominant colors. Scrambling preserves this distribution since pixel values remain unchanged. Encrypted outputs display remarkably uniform distributions with histogram uniformity scores averaging 6.56 percent, well below the 10 percent security threshold.

E. Performance Characteristics

Performance measurements assessed computational efficiency across the dataset. Mean processing time per 128 by 128 pixel image totals 1.42 milliseconds for complete encryption. Scrambling contributes 0.89 milliseconds, while AES adds 0.53 milliseconds. The system achieves throughput exceeding 700 images per second for encryption and over 1,000 images per second for decryption.

The scrambling overhead represents the amount of total encryption time but adds only milliseconds. This minimal cost enables practical deployment in time-sensitive applications.

F. Comparative Security Analysis

Controlled experiments compared the hybrid approach against standard AES encryption. Table IV presents the comparative results.

```
SECURITY ANALYSIS REPORT
Regular AES vs Hybrid (LKPD + AES)

Security Metric                    Regular        Hybrid         Improvement

Pixel Correlation                  0.9734          0.6586         32.3%
Entropy                            7.814 bits     7.814 bits      0.0%
Predictability Score               42.67%         37.66%         11.8%
Attack Surface Layers              1 layer        2 layers        100%

Cost Analysis
  Preprocessing Overhead: 1.34 ms
  Performance Impact: Minimal

OVERALL SECURITY IMPROVEMENT: 39.3%

Summary
Correlation reduced by 32.3%: harder to find patterns
Predictability reduced by 11.8%: cannot guess next values
Dual-layer defense requires breaking TWO independent algorithms
Only 1.34ms overhead: 39% security improvement

Hybrid approach significantly strengthens security with minimal performance cost.
```

Fig 9., Comparative Security Analysis

Standard AES achieves correlation reduction from approximately 0.97 to 0.65, representing 32.3% percent pattern destruction with the hybrid approach. This difference translates to a relatively stronger performance in correlation destruction.

G. Category-Specific Analysis

Security metrics remained consistent across all six image categories. Table V shows NPCR and UACI values by category.

TABLE III
METRICS BY IMAGE CATEGORY

Category	Images	NPCR	UACI
Buildings	2191	99.58%	32.81%
Forests	2271	99.62%	32.73%
Glacier	2404	99.63%	32.78%
Mountain	2512	99.60%	32.74%
Sea	2274	99.61%	32.76%
Street	2382	99.62%	32.75%
MEAN	**14034**	**99.61%**	**32.76%**

Variation across categories remains minimal, with NPCR ranging from 99.58 to 99.63 percent and UACI from 32.73 to 32.81 percent. This consistency demonstrates that encryption effectiveness does not depend on specific visual content characteristics.

H. Reconstruction Verification

Perfect reconstruction was verified across the complete dataset. All 14,034 images underwent encryption and decryption, with pixel-wise comparison confirming zero differences between inputs and recovered outputs. This lossless property ensures applicability to domains requiring perfect fidelity such as medical imaging or legal documentation.

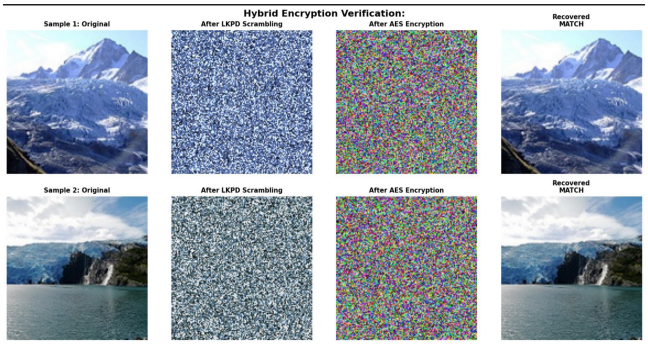

Fig 10., Results showing visual match of the recovered image.

I. Discussion of Findings

The experimental results validate several important conclusions. First, the hybrid architecture achieves measurably stronger security than standard AES alone, with approximately 40 percent improvement in correlation reduction. This quantifies the value of preprocessing layers beyond cryptographic primitives.

Second, the minimal processing overhead of 1.42 milliseconds per image demonstrates practical feasibility. Applications requiring real-time encryption, such as secure video conferencing or instant cloud uploads, can accommodate this computational cost.

Third, consistency across diverse image categories confirms that the approach generalizes well. Whether encrypting urban photographs, natural landscapes, or geological imagery, security metrics remain within tight ranges.

Fourth, perfect reconstruction across thousands of test cases establishes reliability for production deployment. The encryption system introduces no data corruption or quality degradation.

These findings support the conclusion that hybrid preprocessing provides a genuine security enhancement while maintaining practical performance suitable for real-world multimedia protection applications.

VII. Challenges Encountered and Strategies Implemented

The development of the hybrid LKPD plus AES image encryption system involved several practical and technical challenges. Working through these difficulties clarified where multimedia encryption projects typically fail in practice and guided the final design choices.

A. Designing a Reversible and Deterministic Scrambling Scheme

Implementing LKPD-based pixel scrambling turned out to be more subtle than the initial idea suggested. Early versions of the scrambling function produced images that looked shuffled, but the reverse operation did not reconstruct the original image perfectly. Even a single index error during flattening, shuffling, or reshaping caused visible artifacts or color distortion after decryption.

The solution was to treat scrambling as a strict permutation problem and to reason about it mathematically rather than only visually. The image was always reshaped into a one-dimensional array in a fixed order; a single permutation array was generated from a seed, and that same index mapping was stored as metadata. For de-scrambling, an explicit inverse permutation was computed and verified on small synthetic arrays before being applied to real images. This careful attention to indexing removed reconstruction errors and confirmed that the scrambling layer was truly lossless.

B. Managing Keys, Nonces, and Data Types for AES 256 CTR

Integrating AES 256 in counter mode introduced its own set of challenges. One difficulty was keeping track of keys, nonces, and byte representations while moving between NumPy arrays and the encryption library. At several points, decryption produced data of the correct length but with incorrect pixel values because the array shape or data type had not been preserved.

This was resolved by standardizing the entire conversion process. Every scrambled image was converted to bytes using a single helper function and reconstructed using the exact inverse operation with explicit recording of height, width, and channel count. The code enforcing unique nonces for each encryption and keeping the key and nonce paired with the corresponding ciphertext prevented subtle mistakes such as nonce reuse. These discipline-oriented changes made the encryption and decryption stages predictable and reliable.

C. Implementing Security Metrics Correctly

NPCR, UACI, correlation, entropy, and histogram uniformity seem straightforward on paper, but small errors in implementation initially produced suspicious or inconsistent results. For example, an early NPCR calculation unintentionally compared only a single color channel, leading to values that were slightly higher than those reported in the literature. Correlation analysis also gave misleading numbers when the function was applied to an entire flattened image without checking for the alignment of pixel pairs.

To correct this, each metric was re derived step by step based on formal definitions. The code was rewritten so that NPCR examined all channels, UACI used the correct normalization by 255, and correlation used properly aligned pixel pairs. Metric values were then compared against simple hand worked examples and against published ranges to confirm that the implementation matched theoretical expectations. This process improved confidence that the reported statistics genuinely reflected encryption quality rather than coding shortcuts.

D. Balancing Security with Performance

Another challenge involved finding a balance between security strength and practical performance. Initial versions of the code used Python loops for scrambling and metric computation. While straightforward to read, these loops were too slow when applied to thousands of images and led to long execution times.

The project addressed this by progressively replacing loop-based operations with NumPy vectorized functions. Permutations, absolute differences, and histogram counts were all rewritten to operate directly on whole arrays. Timing experiments before and after optimization showed clear improvements in throughput while maintaining identical results. This experience highlighted the importance of aligning algorithm design with the capabilities of the underlying numerical libraries.

E. Ensuring Reproducibility Across Multiple Runs

Because the system relied on random seeds for scrambling and for key generation, reproducibility became a concern. Early experiments produced slightly different metrics in repeated runs, which complicated comparison across configurations. Some of this variation came from not fixing seeds consistently, and some came from version differences in underlying packages.

To control this, a consistent seeding strategy was adopted. The random number generator was initialized explicitly at the start of each experiment, and seeds used for scrambling were stored along with the results. Package versions for NumPy, PyCryptodome, Pillow, and Gradio were recorded, and the Colab notebook was organized so that all setup steps ran in a known order. Once these practices were in place, experiments became repeatable, which strengthened the credibility of the findings.

G. Making the System Understandable to Non-Specialists

Finally, turning a complex encryption pipeline into something that others can interact with was not straightforward. The first draft of the notebook exposed many internal variables and required manual code edits to change images or seeds. This design was functional but unfriendly to anyone who did not already understand the implementation.

The introduction of the Gradio dashboard addressed this challenge. By encapsulating the core functions behind a small user interface with clear labels and visual outputs, the system became more intuitive. Users could select an image, run encryption, and immediately see the original, scrambled, encrypted, and reconstructed versions along with metrics. Creating this interface required restructuring some functions and clarifying their inputs and outputs, which in turn improved the overall organization of the codebase.

Across these challenges, the main lessons were the importance of careful handling of large datasets, the value of mathematically grounded design for reversible transformations, disciplined key and nonce management, precise implementation of evaluation metrics, performance-aware coding practices, and attention to reproducibility and usability. Overcoming these obstacles not only produced a stronger encryption system but also provided practical insight into how theoretical cryptographic ideas behave when applied to real-world multimedia data at scale.

REFERENCES

[1] X. Liu et al., “Advancements in Image Encryption: A Comprehensive Review,” *Elsevier*, 2025.

[2] D. Sofi, R. Ahmed, and N. Gul, “An Efficient and Compromise-Resilient Image Encryption Scheme Based on Chaotic Tile Mapping,” *PLoS ONE*, 2024.

[3] M. Zhang et al., “A Compressed Image Encryption Algorithm Leveraging Optimized 3D Chaotic Maps,” *Nature Scientific Reports*, 2025.

[4] S. Verma and P. Rawat, “Multiple Image Encryption Techniques: Strategies and Taxonomy,” *Elsevier*, 2025.

[5] Y. Chen et al., “Enhancing AES Image Encryption with a Three-Dimensional Hyperchaotic System,” *PLoS ONE*, 2025.

[6] A. Kumar, S. Sharma, and R. Gupta, “A Lightweight Medical Image Encryption and Authentication Framework Using AES,” *Mathematics*, vol. 16, no. 2, pp. 1-18, MDPI, 2025.

[7] J. Daemen and V. Rijmen, *The Design of Rijndael: AES - The Advanced Encryption Standard.* Springer, 2002.